

AI 시스템 아키텍처 (멀티 에이전트 아키텍처)

SKN Final Project 최종 산출물 — CopyLane

팀장 **오한빈**

팀원 **박수진 · 권소라 · 소성민 · 이서은**

작성자 **오한빈** (팀장)

도메인 제안 **권소라**

개정 **2026-10-21** · 기준 커밋 **ksr b54246d**

최종 산출물 · 개정 2026-10-21

목차

1. 한 장 요약 4

2. 그래프 셋과 코어 하나 5

3. 코어 노드 — 무엇을 하고 어디까지 구현됐나 7

3-1. 법별 팬아웃 — 라우팅 규칙 8

4. 판정 — 사전 쪽과 인코더 쪽 9

5. 종착 — 검수 그래프 10

5-1. 그래프 밖 — 검수 화면의 「이 문장 고쳐 쓰기」(sLLM) 10

5-2. 그래프 밖 — 카피 생성 엔진(진입점 B 화면) 11

6. 품목 분기 — 같은 문구가 전제에 따라 갈린다 13

7. 상태 — 그래프가 나르는 것 15

8. 설계 원칙이 구조에 박힌 자리 16

9. 응답시간 — 구조가 정하는 것 17

10. 남은 것 18

작성자 오한빈(팀장) · **개정** 2026-10-21 · **기준 커밋** `ksr b54246d` (main `2f1e86d` 포함 · 1차 작성은 `f8ebb09`) · 예전 제출본 2026-10-06 판(기준 `21e4374` · `c7f483a` · `b0ad791`)

이 문서가 다루는 것 — ① 판정 그래프의 구조(노드 · 엣지 · 상태) ② 노드마다 무엇이 구현됐고 무엇이 비어 있는가 ③ 품목 분기와 종착 ④ 그래프 밖에서 같은 판정 코어를 부르는 생성 장치(카피 생성 엔진 · sLLM 고쳐 쓰기)가 그래프와 어떻게 이어지는가 ⑤ 설계 원칙이 구조에 박힌 자리

다루지 않는 것 — 판정 인코더의 모델 구조와 학습(학습결과서 · 학습 모델 명세), sLLM 학습(자체 sLLM 결과서), 화면 구성(화면설계서), 시스템 전체 구성 · 배포(시스템 구성도 · 배포계획)

정본 — 구조는 코드(`app/graph.py`의 노드 순서 표 `CORE_BEFORE_LAWS` · `CORE_AFTER_LAWS`와 각 노드, `app/contracts.py`)가 정본이고 이 문서는 그것을 읽어 옮긴 것이다. 수치는 `docs/00_사실원장.md`(이하 「원장」)가 정본이며 인용에 날짜 · 절 · 기기를 함께 적는다. 이번 개정에서 노드 · 엣지는 `build_core()` · `build_review()` 컴파일본을 직접 열어 확인했다(기준 커밋 사본 · LangGraph 1.2.11 · 이 PC).

1. 한 장 요약

물음	답 (2026-10-21)
어떤 꼴의 멀티 에이전트인가	**파이프라인형** — 도구 호출형(ReAct · MCP)이 아니다. 판정 그래프 안에는 외부 도구 호출이 없다(D-11). 카피 생성 엔진의 GPT 호출은 그래프 밖(화면 BFF)에 있다(§5-2)
에이전트에 해당하는 것	법마다 하나씩 선 **법별 판정 노드 셋**(표시광고법 · 식품표시광고법 · 화장품법)과 그 앞뒤의 공통 노드
협업 방식	**팬아웃 – 팬인** — 검색 · 사전 매칭 · 인코더는 앞에서 한 번 돌고, 품목에 맞는 법 노드로 동시에 보낸 뒤 한 곳에서 모은다(D-267)
그래프 수	판정 코어 서브그래프 1 + 진입점 그래프 3. 실제로 도는 것은 **검수 그래프(진입점 A)** 하나다. 생성 그래프(B)는 스텁이고 앱에서 부르지 않으며, 조립(C)은 상태만 정의돼 있다
판정 코어	**하나** — 검수 · 분기 선택 뒤 다시 판정 · 고쳐 쓴 문구의 재판정 · 카피 생성 후보 판정 · AI 광고 생성 칸 판정이 모두 같은 함수(`app.api.judge`)로 같은 그래프를 지난다(D-266 · D-119)
지금 도는 것	검수 경로 전체 — 사전 판정 + 인코더(D4-20) 근거 + 위험도 하한 + 품목 분기 + 사실 · 실증 질문 + 참고 조문 · 그래프 밖: 카피 생성 엔진(GPT), 검수 화면의 「이 문장 고쳐 쓰기」(sLLM 서버가 있을 때), AI 광고 생성 화면
지금 비어 있는 것	문서 단위 규칙(R-D1) · 품목을 문구로 판별하는 일(`classify`) · 생성 그래프(B)의 노드 · 코어 `POST /generate` · `/compose` (501) · 리랭커
틀	LangGraph 1.2.11 · FastAPI 0.141.1 · Python 3.11 · PostgreSQL 16.15 + pgvector 0.8.6 · 인코더 `copylane-encoder-kcbert-D4-20-기능성편입` · 임베딩 `nlpai-lab/KURE-v1` (1024차원)

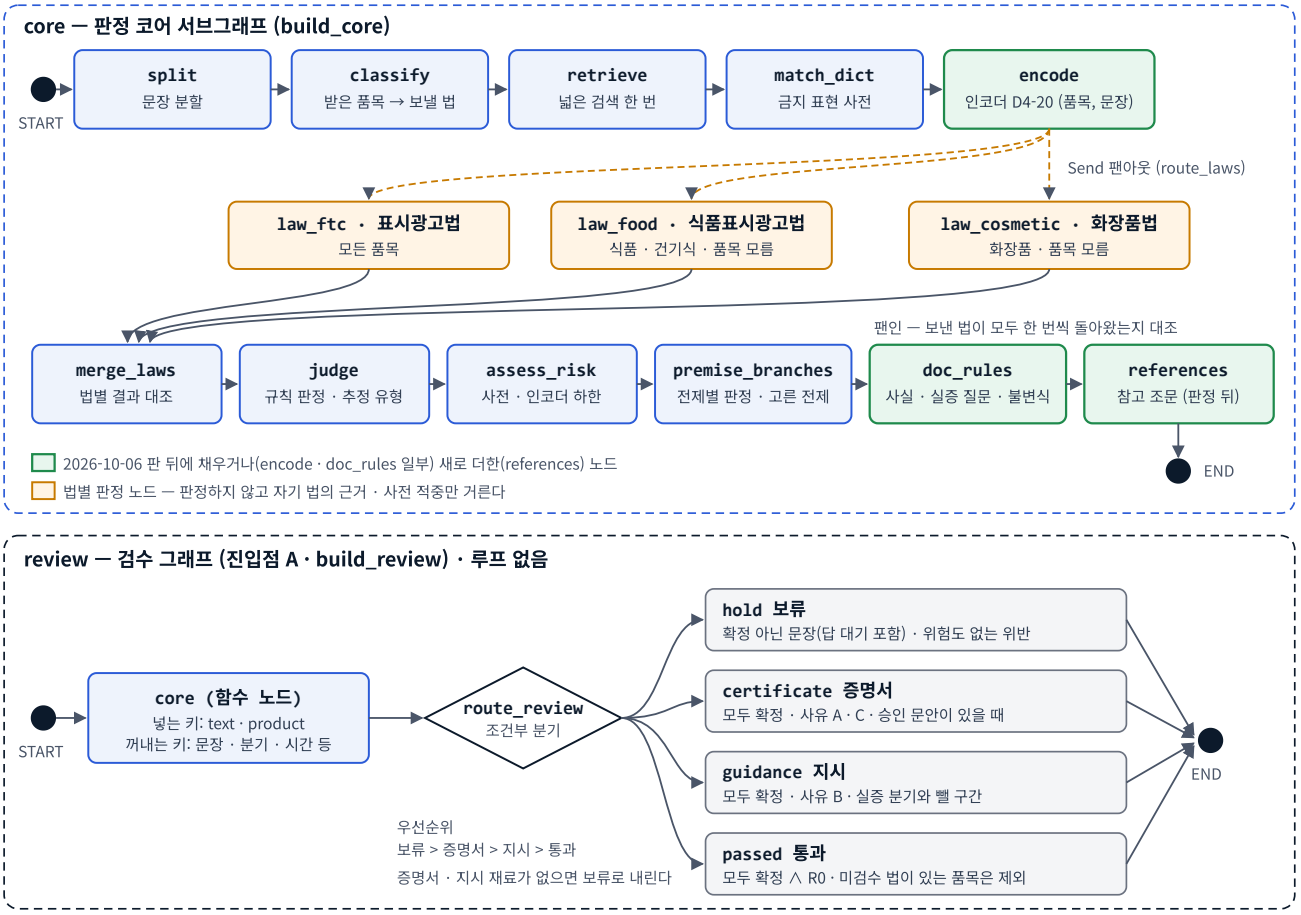
10-06 판과 달라진 큰 줄기는 다섯이다.

1. **`encode` 노드가 채워졌다** — 잠금 채택된 인코더 D4-20이 문장마다 유형 확률을 내고, 그 신호가 판정 · 위험도 · 질문에 들어간다(§3 · §4).
2. **`doc_rules` 노드에 일이 생겼다** — 사실 · 실증 질문(조건부 위반)과 기록 · 분기 불변식 검사를 여기서 한다. 품목을 모르면 기록 문장의 질문을 걷어 「고르기 대기」로 두는 일도 여기서 한다(10-21). 문서 단위 규칙 자체는 아직 없다(§3 · §6).
3. **`references` 노드가 더해졌다**(10-19) — 참고 조문을 판정 뒤에 유형 · 법 안에서 고른다(§3).
4. **사용자가 고른 전제는 그래프의 입력이다**(10-21) — 고르면 그 전제 하나로 다시 판정하고, 답하기 전 조건부 위반은 보류(답 대기)로 둔다(§5 · §6).
5. **진입점 B는 그래프가 아니라 화면의 카피 생성 엔진이 맡는다** — GPT 후보 → 코드 필터 → 판정 그래프 전체(§5-2).

2. 그래프 셋과 코어 하나

```
core     (서브그래프)  split → classify → retrieve → match_dict → encode
                       → [법별 팬아웃: law_ftc · law_food · law_cosmetic]
                       → merge_laws → judge → assess_risk → premise_branches → doc_rules → references
review   (진입점 A)   START → core → route_review → certificate · guidance · hold · passed → END   ← 루프 없음
generate (진입점 B)   keyword_screen → assemble → claim_ledger → rejudge → (루프) → frontier · search_failed
                       ← 스텁 · 앱에서 부르지 않음. 실제 카피 생성은 app/routers/copygen (§5-2)
compose  (진입점 C)   상태만 정의 — 그래프 없음
```

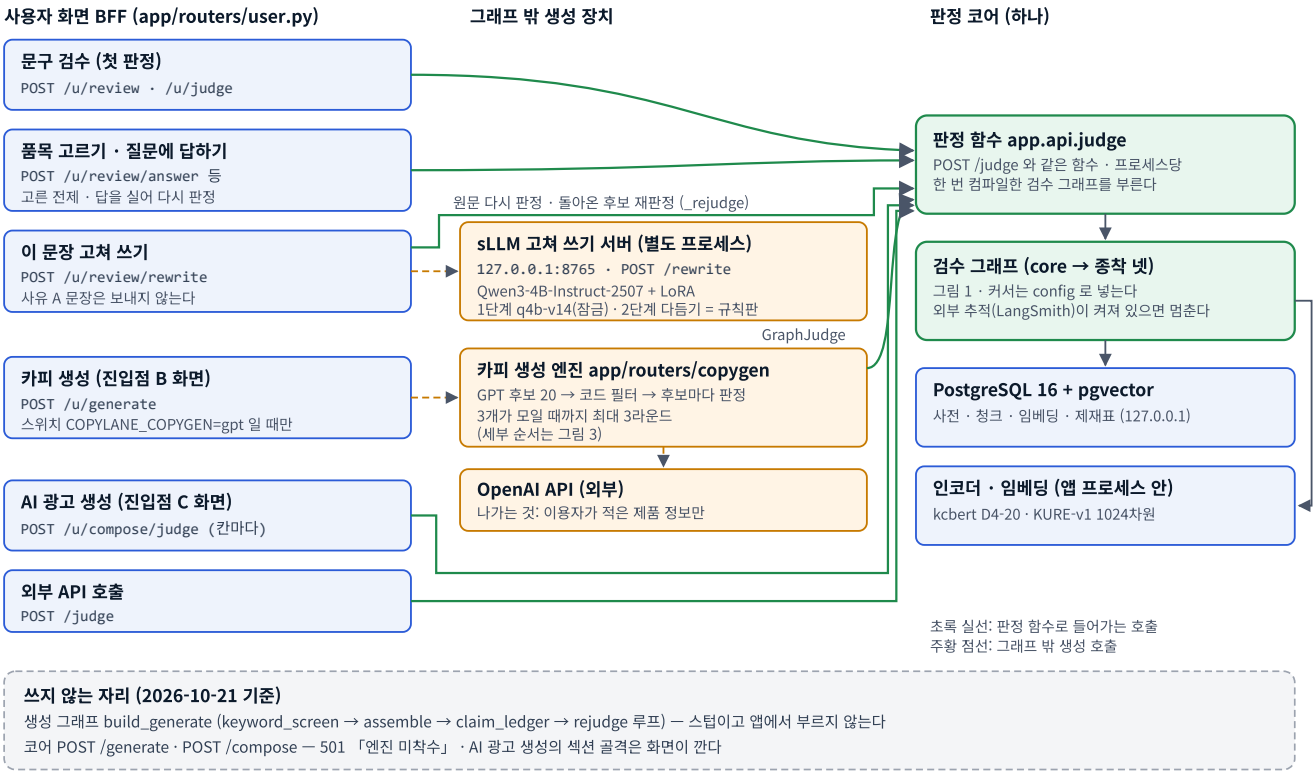
그림 1. 판정 코어 서브그래프와 검수 그래프 (app/graph.py · ksr b54246d 컴파일본에서 확인)



- **코어를 하나만 두는 까닭** — 검수가 내는 판정과 생성 쪽이 자기 후보를 다시 판정할 때의 판정이 같은 코드여야 한다. 지금은 생성 그래프 대신 화면의 생성 장치들이 같은 판정 함수를 부른다(그림 2).
- **검수 그래프에 루프가 없는 까닭** — 검수는 문구를 만들지 않는다(D-265). 사용자가 품목을 고르거나 질문에 답하면 화면이 그래프를 **처음부터 다시 부른다**. 그래프 안에서 되도는 것이 아니다(D-348 보충 34 결정 2).
- **코어를 노드로 끼우는 방식** — 컴파일한 서브그래프를 그대로 끼우면 부모의 누적 키가 두 번 쌓였다(09-23 실측). 그래서 넣을 키(`CORE_IN` = 원고 · 제품 정보)와 꺼낼 키(`CORE_OUT` = 문장 · 보낼 법 · 근거 · 사전 · 인코더 신호 · 법별 결과 · 문장 판정 · 분기 · 시간)를 표로 든 함수 노드가 코어를 부른다.

- **실행** — 판정 함수(`app.api.judge`)는 검수 그래프를 프로세스당 한 번 컴파일해 두고, DB 커서를 `config` 로 넣어 부른다. 노드가 스스로 DB에 붙지 않는다. 그래프를 지을 때 외부 추적(LangSmith)이 켜져 있으면 광고 원문이 밖으로 나가므로 실행을 거부한다(D-43).

그림 2. 진입점과 판정 코어의 연결 — 모든 판정 · 재판정은 같은 함수 하나를 지난다



3. 코어 노드 — 무엇을 하고 어디까지 구현됐나

순서는 `app/graph.py` 의 `CORE_BEFORE_LAWS` (팬아웃 앞 5) · `LAW_NODES` (법별 3) · `CORE_AFTER_LAWS` (팬아웃 뒤 6)이고, 컴파일본과 스텁 한 바퀴(`run_review_stub`)가 이 표 하나를 쓴다. 상태 표시: ✅ 구현 · ◐ 일부.

순서	노드	하는 일	상태 (2026-10-21)
1	`split`	원고를 문장으로 나눈다(`app/sentsplit.py` · 문장은 원문의 부분 문자열이라 원문 좌표를 되찾는다)	✅ · 분할 정확도는 미측정 (정답 셋 없음)
2	`classify`	받은 품목으로 **보낼 법**을 고른다 — 품목이 없으면 세 법 전부	◐ 받은 값만 쓴다 · 문구로 품목을 판별하는 일은 없다 (D-348 보충 18 결정 1 미집행)
3	`retrieve`	조문 검색 — 벡터 · 어휘 두 갈래를 법마다 폭만큼 넓게 **한 번** 받는다 (`rt.wide` · D-291). 결과는 참고 조문의 재료이고 판정 근거가 아니다	✅
4	`match_dict`	금지 표현 사전 매칭 — 단독판정 항목 가운데 불가 사유 칸이 C인 것만 확정 재료로, 나머지는 유형 후보로 낸다(보충 16)	✅
5	`encode`	**인코더** — 문장마다 9개 위반 유형의 확률과 후보. D4-20은 (품목, 문장) 쌍을 받으므로 필요한 품목 입력(모름 · 식품 · 건기식 · 화장품)을 한 배치로 돌린다. 잠금(`models.lock.json`)에 없는 판은 올리지 않는다	✅ **10-07 배선 · 10-09 응답 반영 · 10-14 품목 입력**
6	`law_ftc` · `law_food` · `law_cosmetic`	**법별 노드** — 자기 법의 검색 후보와 자기 법을 인용한 사전 적중만 거른다. 판정하지 않는다. 화장품법 노드는 식품 축의 질병 · 의약품 절대형 적중을 화장품법 제13조①1호로 옮겨 받는다(보충 23)	✅
7	`merge_laws`	팬인 — 보낸 법이 모두, 한 번씩, 문장을 다 보고 돌아왔는지 대조한다. 어긋나면 멈춘다	✅
8	`judge`	문장 판정(사전 쪽) + 인코더 유형 후보를 문장의 `encoder_candidates` 칸에 싣는다(전제에 따라 거르거나 옮김). 주의 · 상담 문구는 「판정 대상 아님」으로 낸다	✅
9	`assess_risk`	위험도 — 사전 하한(서명된 제재표 뷰)과 인코더 하한을 **칸을 나눠** 적는다. 주된 광고법을 보지 않은 품목(일반상품 · 전용법 미수록)의 「걸린 것 없음」은 보류(`law_uncovered`)로 내린다	✅
10	`premise_branches`	품목 분기 — 전제마다 판정을 미리 계산한다. 사용자가 전제를 골랐으면 그 전제 하나로 판정한다(분기 없음 · 보충 34)	✅
11	`doc_rules`	사실 확인 · 실증 질문(조건부 위반)과 인코더 쪽 판정을 문장 · 분기마다 정하고, 기록 판정이 분기와 맞는지 불변식으로 본다(보충 35). 품목을 모르면 기록 문장의 질문은 걷고 고르기 대기로 둔다 — 질문은 분기 미리보기에만 남는다(보충 37 결정 2 · `_pick_first`)	◐ 질문 · 불변식은 구현 · **문서 단위 규칙(R-D1)은 비어 있다**
12	`references`	(10-19 추가) 참고 조문을 판정 뒤에 문장의 (유형 · 법) 안에서 다시 고른다. 판정 칸(판정 · 유형 · 근거 조문 · 위험도 · 분기)은 건드리지 않는다	✅

- **빈 자리는 값을 지어내지 않는다** — 비어 있는 노드는 상태를 그대로 넘긴다. DB가 없으면 사전을 못 훑었다는 사실 (`ran=False`)을, 인코더가 설정되지 않았으면 신호 없음을 그대로 남긴다. 「못 봤다」가 「안 걸렸다」로 바뀌지 않게 하려는 것이다(D-220).

- **인코더가 설정됐는데 올리지 못하면** 경고를 남기고 규칙 판정만으로 나간다. 응답의 판 표지(`judged_by`)에는 인코더가 실제로 돌았을 때만 `+encb:<판 지문>` 이 붙는다(예: `rule-0.3.0-dict+encb:c451942436f0`). 앞머리 `rule-0.3.0-dict` 는 이전 결과와 이어 보려고 이름만 그대로 둔 것이고, 지금은 인코더 근거가 판정에도 들어간다(§ 4).

3-1. 법별 팬아웃 — 라우팅 규칙

품목(ProductContext.category)	보내는 법 노드
식품 · 건기식	`law_ftc` (표시광고법) + `law_food` (식품표시광고법)
화장품	`law_ftc` + `law_cosmetic` (화장품법)
품목 모름(`None`)	셋 다
일반상품 · 전용법 미수록(의료기기 · 의약외품 등)	`law_ftc` 만 — 통과를 내지 않고 「미검수 법」 고지를 단다(D-271 · D-277)

- 위 표는 `laws_for()` 를 품목마다 직접 불러 확인했다. 표시광고법은 범위가 「모든 품목」이라 팬아웃이 빌 수 없고, 그래도 비거나 모르는 이름이 오면 `route_laws` 가 멈춘다(빈 `Send` 는 LangGraph가 오류 없이 그래프를 끝내기 때문이다).
- 검색 · 사전 · 인코더는 팬아웃 **앞에서 한 번** 돈다. 법마다 다시 부르면 비용이 법 수만큼 는다(D-267).
- 법별 노드는 `law_payload()` 가 보낸 키(문장 · 제품 정보 · 근거 · 사전 적중 · 참고 범위)만 본다. 컴파일본의 `Send` 와 스텁이 같은 함수를 쓴다.
- 건강기능식품법은 노드가 없다 — 참고 전용이라 위반 근거로 보내지 않는다(D-271 ⑥). 법별 노드 ↔ 법 축 대응표가 어긋나면 모듈을 읽을 때 멈춘다.

4. 판정 — 사전 쪽과 인코더 쪽

10-06 판의 판정은 「사전 적중 = 확정 · 그 밖은 보류」 둘뿐이었다. 지금은 **사전 쪽과 인코더 쪽을 따로 판정하고** 기록에서 합친다(D-348 보충 5). 문장 하나의 판정 칸은 여전히 넷 중 하나다.

판정	언제
미판정	사전을 훑지 못했다 — 못 본 것을 본 것처럼 말하지 않는다
확정	(사전 쪽) 이 판정에 보낸 법을 인용한 칸 C 단독판정 항목이 걸렸다 · (인코더 쪽) 품목이 정해졌고 그 법 축 칸이 C인 유형의 확신이 기준값 이상이다 · (답) 사용자가 질문에 답해 정해졌다 · 위반이 없는 확정(예: 판정 대상 아님 · 답으로 풀림)도 있다
근거 없음	유형은 잡았는데 인용 좌표를 세우지 못했다(D-224)
보류	그 밖의 전부 — 확신 부족 · 품목 선택 필요 · 전제 선택 필요 · 미검수 법 · **답 대기(조건부 위반)**

사전 쪽

- 확정은 「사전의 법 · 호 근거 ∧ 단독판정 자격 ∧ 칸 C」일 때만이다(보충 16). 주장이 아닌 자리(고지문 · 부정문 등)의 적중은 확정 막기 가드가 후보로 내린다(`claimscope`).
- 단독판정 자격이 없는 항목이 울리면 보류 문장에 유형 후보만 싣는다. 확정하지 않는다.

인코더 쪽 (사전 쪽 판정이 없는 문장)

- 유형 후보는 문장의 `encoder_candidates` 칸에 실린다. 화면에는 「AI 추정 유형 있음 — 판정 아님」으로 보인다(D-345).
- 전제에 따라 후보를 거르거나 옮긴다 — 건기식(인정 · 비인정) · 일반식품 기능성 전제에서는 건강기능식품 오인이 서지 않고, 건기식 비인정 전제에서는 건강기능식품 오인 후보를 [별표 1] 4.나(거짓_과장)로 옮기며, 화장품 전제에서는 질병 표방 · 건기식 오인을 의약품 오인으로 옮긴다(D-323 결정 6 · D-338).
- **절대형(칸 C)** 유형은 품목이 정해졌을 때만 답 없이 확정한다. 확신 기준값 τ는 판에 딸린 값이다 — D4-20에서 절대형 0.699, 전제가 답인 건강기능식품 오인 0.693(원장 10-19 ④ · ⑩). 품목을 모르면 확정하지 않고 「품목 선택 필요」 보류로 둔다(보충 8 · 31).
- **실증형(칸 B)** · 사실에 달린 유형은 답 없이 확정하지 않는다. 문구 밖 사실(수상 · 인증 · 비교 기준 · 실증 자료 등)을 묻는 질문을 달고 보류로 둔다. 사용자의 답은 세 가지다 — 「예」면 확정 · R0(출처 = 사용자 답 · 검증 아님) · 「해당 없음」이면 그 질문을 걷음 · 「아니오」면 확정 위반.
- 인코더만 잡은 거짓 · 과장의 확정 기준값은 아직 정하지 않았다(`ENCODER_CONFIRM_TAU = None` · 닫힘).

공통

- 불가 사유는 **(위반 유형 × 법 축) 칸**으로 정한다(보충 6) — A 자격형(인정 · 심사를 받으면 가능) · B 실증형(근거 자료가 있으면 가능) · C 절대형(길이 없음). 같은 비방광고도 식품표시광고법 · 화장품법 축은 C, 표시광고법 축은 B다. 칸이 없으면 사유를 모르는 것이고 보류다.
- 위험도 = 사전 하한과 인코더 하한의 최댓값. 인코더 근거에도 대응표의 호를 붙여 근거 조문이 선다(보충 2).
- 인코더 라벨은 9유형이다 — 질병_예방치료_표방 · 건강기능식품_오인 · 의약품_오인 · 거짓_과장 · 소비자_기만 · 후기_체험기_기만 · 부당_비교광고 · 비방광고 · 기능성화장품_오인. 계약 `Violation` 은 여기에 추천_보증_뒷광고(범위 밖) · 실증책임_위반(후보)을 더한 11값이다.

5. 종착 — 검수 그래프

`route_review`의 규칙(우선순위 **보류 > 증명서 > 지시 > 통과** · D-268 · D-125).

종착	언제 (코드 순서대로)	사용자가 받는 것	상태
`hold` 보류	문장이 없다 · 확정이 아닌 문장이 하나라도 있다(답 대기인 조건부 위반 포함 · 보충 33) · 위험도 없는 확정 위반이 있다	문장별 판정 · 보류 사유 · 분기 · 질문	✅
`certificate` 증명서	모두 확정 · 사유 A나 C가 있다 · 그 (품목의 법, 유형, 사유) 조합의 승인 문안이 있다	「합법화 불가」 사유 설명 · 자격형이면 자격 안내	✅ 문안이 없으면 보류로 내린다
`guidance` 지시	모두 확정 · 사유 B만 있다 · 실증형 문장마다 실증 분기와 빨 구간이 있다	빨 구간 · 필요한 실증 자료	✅ 재료가 모자라면 보류로 내린다
`passed` 통과	모두 확정 ∧ R0 · 미검수 법이 있는 품목(일반상품 · 전용법 미수록)이 아니다 · 품목을 모르면서 판정 대상 아님 문장이 섞여 있지 않다	통과(법령 위반 소지가 검출되지 않았다는 뜻 · 안전 보장이 아님)	✅

- **답하기 전은 답 대기다**(10-21 보충 33). 종전에는 답하지 않은 조건부 위반을 가장 나쁜 쪽(확정 B)으로 세어 증명서 · 지시로 보냈다 — 사용자가 답하지 않은 사실을 가정한 판정이었다. 지금은 확정이 아니면 모두 보류이고, 가장 보수적인 쪽은 위험도에만 남긴다.
- **지시 종착의 재료**는 10-18에 사전 쪽 실증 분기를 걷은 뒤(보충 20) 인코더 쪽 실증 답뿐이다.
- 증명서 사유 문안은 (법, 위반 유형, 불가 사유)를 열쇠로 `app/reasons.py`의 승인된 줄에서만 꺼낸다. 품목 미확정 · 일반상품 · 전용법 미수록은 꺼낼 법이 없어 증명서가 없다.
- **측정된 종착 분포**(원장 10-21 ⑤ · 커밋 `01287d6` · 팀장 집 기기 Windows · 확장 dev 조건부 1,132 행 · 게이트 아님) — 종착 보류 1,011 · 증명서 68 · 통과 53. 판정을 내린 행 66 기준 selective risk 1.5 %(95 % 구간 0.3 ~ 8.1 %) · coverage 8.4 %(66 / 채점 783). 그래프의 봉인 평가는 아직 하지 않았으므로 모두 dev 수치다. 그 뒤 코드(`49aafce` · §6의 기록 질문 걷기)로는 품목을 주지 않은 무조건부만 다시 쟀고(§6), 조건부는 다시 재지 않았다 — 조건부 수는 위 10-21 ⑤ 가 최신이다.
- 생성 그래프의 종착(프론티어 · 탐색 실패)은 계약과 라우터만 있다. 노드 여섯 가운데 `keyword_screen` · `claim_ledger` · `rejudge`는 빈 노드이고, `assemble`은 라운드 수만 세며, `frontier` · `search_failed`는 종착 값만 낸다. `build_generate()`는 게이트 시험만 짓고 앱에서는 부르지 않는다. 이 그래프를 채울지, 진입점 B의 정의를 화면 엔진으로 고칠지는 판정 대기다.

5-1. 그래프 밖 — 검수 화면의 「이 문장 고쳐 쓰기」(sLLM)

항목	내용 (2026-10-21)
결정	검수 화면의 고쳐 쓰기는 이서은의 자체 sLLM이 맡는다(D-332 결정 3 · 10-13 확정). 판은 잠금됐다(D-335)
서버	`docs/lse/sllm_service.py --serve --port 8765` · `POST /rewrite` · GPU용 가상환경(`.venv-sllm`)에서 앱과 **별도 프로세스**로 띄운다. 앱(`.venv`)은 CPU torch라 한 프로세스에 합치면 패키지가 부딪힌다
모델	베이스 `Qwen/Qwen3-4B-Instruct-2507`(apache-2.0) + LoRA · 1단계 `copylane_sllm_lora_adapter_q4b-v14`(롤백 v13 · `models.lock.json`) · 2단계 다듬기의 기본은 **규칙판**(`docs/lse/polish_rule.py` — 모델을 쓰지 않고 제품 모양별 쓰는 법 문구를 붙인다 · D-342 · 서버 코드 `POLISH_MODE = "rule"`). 잠금의 2단계 어댑터 `copylane_sllm_persona_adapter_q4b_polish`(롤백 persona_q4b)는 모델판 다듬기의 되돌림 · 고객층 말투 전용이고 앱 경로는 지금 부르지 않는다. 규칙판 채택 조건 다섯의 집행은 결정기록에 「대기」로 적혀 있다

항목	내용 (2026-10-21)
클라이언트	`app/routers/sllm_client.py` — 기본 `http://127.0.0.1:8765`, `COPYLANE_SLLM_URL=off`(또는 빈 값)이면 끈다. 표준 라이브러리 `urllib`로 부르고 대기 상한은 90초다

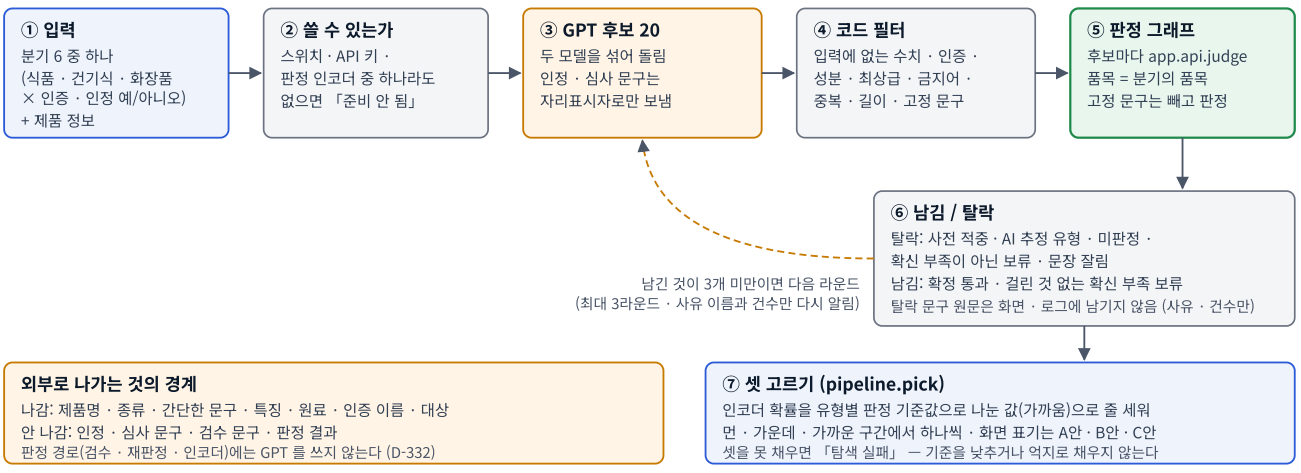
흐름(`app/routers/user.py` `_rewrite_sentence`)

1. 화면이 지적 문장 하나를 고른다. 품목 분기가 있는 문구는 **분기를 고른 뒤에만** 고쳐 쓴다. 자격형(A) 문장은 보내지 않는다 — 표현이 아니라 자격의 문제라 같은 위반을 되풀이한다(D-59).
2. 위반 유형은 서버(앱)의 판정 결과에서 읽는다(폼을 믿지 않는다). 사전 유형이 없으면 인코더 추정 유형으로 부른다.
3. sLLM 서버가 문장 · 위반 유형 · 품목을 받아 후보(`candidate`) · 보류(`hold`) · 고칠 수 없음(`infeasible`) 중 하나를 돌려준다. 다듬은 문장과 담백한 1단계 문장을 함께 줄 수 있다.
4. 돌아온 문장을 **앱의 판정 함수로 다시 판정**한다(`_rejudge` → `app.api.judge`). 고른 전제가 있으면 그 전제의 제품 정보(품목 · 전제)를 실어 판정한다(보충 37 결정 5). `_rejudge`에는 앞서 한 답(사실 · 실증)을 넘길 칸도 있지만, 고쳐 쓰기 경로(`_rewrite_sentence`)는 아직 답을 넘기지 않는다 — 고친 문장의 재판정에는 고른 전제만 실리고, 답을 넘기는 화면 쪽 배선은 대기다. 다듬은 안이 재판정에서 떨어지거나 다듬은 안에만 AI 추정 유형이 남으면 담백한 안을 올린다.
5. 「인정 건강기능식품에 한해 표시」 조건이 붙은 후보는 고른 전제에서 그 조건이 채워질 수 없으면 내지 않는다(`_note_conflict`).

- 검수 응답(`JudgeResponse`)의 후보 목록은 비어 있다 — 고쳐 쓰기는 판정과 따로 부르는 호출이고, 화면이 그 결과를 판정 옆에 붙여 보일 뿐이다. 그래서 「검수는 대체 문구를 내지 않는다」(D-265)와 충돌하지 않는다.
- 서버가 없거나 응답이 깨지면 후보를 그리지 않는다. 「이 환경에 두지 않음(off)」과 「연결 실패(down)」를 나눠 안내한다.
- 재판정 결과 「위반을 찾지 못함」은 통과가 아니다. 종착이 통과일 때만 통과로 표시한다.
- 배포에 넣을지는 앱 코드에서 「미정」으로 적혀 있다(`sllm_client.py:24`). 배포 담당 브랜치(origin/ssm, 아직 ksr에 없음)의 기록에는 서버에서 sLLM 실행 확인(10-19)이 적혀 있고, 서버에서의 한 문장 시간은 측정 전이다.
- 새 판 q4b-n2는 채택 결정(10-17)이 됐지만 잠금 등재 전이라 서버는 v14를 싣는다.

5-2. 그래프 밖 — 카피 생성 엔진(진입점 B 화면)

그림 3. 카피 생성 엔진의 한 번 실행 (app/routers/copygen · pipeline.run)



「남김」은 위반 후보를 찾지 못했다는 뜻이다 — 적법 확인이 아니다. 계약 GenerateResponse(잔여 리스크 · 소구력)는 채우지 않는다.

항목	내용 (2026-10-21)
위치 · 소유	`app/routers/copygen/` (branches · filters · judge · llm · pipeline · errors) · 권소라 · 화면 `POST /u/generate`
결정	생성 경로에만 GPT를 허용한다 · 판정 경로(검수 · 재판정 · 인코더)는 GPT 금지 · 밖으로는 이용자가 넣은 제품 정보만 보낸다 — D-332(10-13 확정 · 팀장). D-78(GPT 런타임 금지)의 예외다
켜는 조건	`.env` `COPYLANE_COPYGEN=gpt` ∧ API 키 ∧ 판정 인코더 설정 — 하나라도 없으면 생성하지 않고 화면은 「준비 안 됨」을 그린다. 키가 있다는 것만으로는 켜지지 않는다
생성 모델	기본 `gpt-5.6-luna,gpt-6-luna` (쉼표로 이으면 섞어 돌림 · `COPYLANE_COPYGEN_MODEL` 로 바꿈) · 호출 대기 상한 90초
분기	6 — 식품(인증 아니오 · 인증 예) · 건강기능식품(인정 예 · 인정 아니오) · 화장품(일반 · 기능성 심사/보고). 건강기능식품 「인정 아니오」 분기는 품목 「식품」으로 판정한다. 인정 · 심사 문구가 필요한 분기(건강기능식품 인정 예 · 화장품 기능성)는 그 문구를 자리표시자로 두고 코드가 끼운다
한 라운드	GPT 후보 20 → 코드 필터 → 후보마다 판정 그래프 → 남김 / 탈락 · 남긴 것이 3개 미만이면 다음 라운드(최대 3라운드 · 최대 60 후보)
판정	`GraphJudge` — 후보 하나하나를 `app.api.judge` (검수 그래프 전체)에 분기의 품목 · 인정 여부를 실어 넣는다. 그래프가 인코더를 돌리지 못했으면 생성하지 않는다

- **코드 필터**(`filters.py`)는 판정 엔진이 못 잡는 것 — 입력에 없는 사실을 지어낸 문장 — 을 먼저 거른다. 사유는 길이 범위 밖 · 중복 · 입력에 없는 수치 · 입력에 없는 인증 · 시험 · 입력에 없는 성분 · 비교 · 최상급 · 분기 금지어 · 고정 문구 자리 이상 · 고정 문구가 앞뒤 말과 이어짐 · 판정엔진 미통과다. 표면 형태만 보므로 말로 풀어 쓴 새 주장은 지나갈 수 있다.
- **남기는 기준** — 그래프는 사전이 침묵한 문장에 통과가 아니라 확신 부족 보류를 낸다(D-269). 그래서 「통과만 남김」이 아니라 「걸린 것을 떨굼」이다 — 사전 적중 · AI 추정 유형 · 미판정 · 확신 부족이 아닌 보류 · 문장 잘림은 탈락, 확정 통과와 걸린 것 없는 확신 부족 보류는 남긴다. 「남김」은 위반 후보를 찾지 못했다는 뜻이지 적법 확인이 아니다.
- **인정 · 심사 문구는 판정에서 뺀다** — 인정받은 글자 그대로라 유형 후보로 걸리면 인정 제품의 광고가 전부 막힌다(10-03 실측). 대신 그 문구 하나를 그 분기의 판정 전제로 따로 판정해 질병 표방 · 의약품 오인이 걸리면 경고한다(D-341 결정 1). 경고일 뿐 후보를 떨구지 않는다.
- **셋 고르기** — 남긴 후보를 「인코더 확률 ÷ 그 유형의 판정 기준값」의 최댓값(가까움) 순으로 세워 먼 · 가운데 · 가까운 구간에서 하나씩 고른다. 위험도 등급이 아니다. 화면 표기는 A안 · B안 · C안이고, 셋을 못 채우면 「탐색 실패」로 낸다.
- **로그** — 탈락 문구 원문 · 이용자 입력은 화면 · 로그에 남기지 않고 사유 유형과 건수만 남긴다. 다음 라운드 프롬프트에도 탈락 원문은 넣지 않는다.
- **계약** — 계약 `GenerateResponse` (잔여 리스크 · 소구력 보존)는 채우지 않는다. 그 수치를 재는 노드가 없다.
- **검수 → 생성 넘기기**(`POST /u/generate/from-review`) — 검수에서 고쳐 쓸 수 없던 문장을 생성 입력 화면에 미리 채워 열 뿐이고, 생성은 부르지 않는다. 위반 구간은 넘기지 않는다.
- **AI 광고 생성**(`/u/compose`) — 문구를 칸마다 `POST /u/compose/judge` 로 같은 판정 함수에 넣고, 결과를 「얹음 · 확인 필요를 달고 얹음 · 얹지 않음」 셋으로 읽는다(`_lands`). 코어 `POST /compose` 는 501이다.

6. 품목 분기 — 같은 문구가 전제에 따라 갈린다

같은 문구가 일반식품이면 위반이고 인정받은 건강기능식품이면 아닐 수 있다. 품목을 모를 때 한 가지 답을 지어내지 않고 **전제마다의 판정을 미리 계산해** 응답에 싣는다(D-263 · D-319). 전제는 여섯이다(`app/premise.py`).

품목	전제 (PREMISES_OF)
모름	식품 · 일반식품_기능성 · 건기식_인정 · 건기식_비인정 · 화장품 · 일반상품 (여섯)
식품	식품 · 일반식품_기능성
건기식	건기식_인정 · 건기식_비인정
화장품 · 일반상품	하나뿐 — 분기 없음
전용법 미수록	없음 — 판정할 법이 없다

상황	기록되는 판정	분기
품목 모름 · 모든 전제에서 같은 판정	그 판정	전제 여섯(늘 낸다)
품목 모름 · 전제마다 다름	보류(`cat_unknown` · 「품목 선택 필요」)	전제 여섯
품목이 식품 · 건기식이고 전제마다 다름	보류(`premise_unknown` · 「전제 선택 필요」)	그 품목의 전제 둘
품목이 주어지고 전제마다 같음	그 판정	접어 둔다(`folded`) — 질문까지 정한 뒤 전제끼리 갈리면 펼친다(보충 32)
사용자가 전제를 골랐다 (`ProductContext.premise`)	그 전제 하나의 판정	없음(보충 34)

- **고른 전제는 입력이다**(10-21 보충 34 결정 1). 종전에는 품목의 전제를 모두 계산하고 화면이 그중 하나를 골라 읽었다. 그러면 답을 넣어 다시 판정할 때 하위 전제가 다시 미정으로 돌아갔다. 지금은 `pm.product_of(전제)` 가 품목 · 자격 · 전제를 함께 실어 그 전제 하나로 판정하고, 분기에서 읽은 결과와 전제를 넣은 결과가 같은 함수(`_premise_sentences`)에서 나온다.
- **답한 뒤에도 전제끼리는 합치지 않는다**(보충 33 · 34 결정 3) — 모든 분기가 같은 판정일 때만 기록을 그 판정으로 두고, 갈리면 전제 보류로 둔다. 위험도만 분기 최댓값을 쓴다.
- **불변식 검사**(10-21 보충 35) — `doc_rules` 끝에서 「기록이 확정이면 모든 분기가 같은 위반으로 확정」인지 본다. 어긋나면 운영에서는 전제 보류로 내리고 로그(`copylane.invariant`)를 남기며, 시험 환경에서는 거부한다. 로그에는 문장 글자를 싣지 않는다.
- **품목을 모르면 기록에는 질문을 싣지 않는다**(10-21 보충 37 결정 2 · `doc_rules` 끝의 `_pick_first`) — 묻는 순서는 품목 → 하위 전제 → 사실 → 실증이다(보충 34 결정 2). 그래서 품목을 모르는 기록 문장이 보류이고 사실 · 실증 질문을 달고 있으면 질문을 걷고 「품목 선택 필요」(`cat_unknown` · 고르기 대기)로 둔다. 분기 미리보기 안의 질문(「이 품목이면 이걸 묻는다」)과 유형 · 위험도 · 근거는 그대로이고, 확정은 손대지 않는다. 분기가 없는 경로에서는 부르지 않는다.
- 재측정(원장 10-21(A) ⑥ · 클론 A 회사 기기 · 팀장 실행 · 커밋 `49aafce` · 무조건부 1,147 행 · 게이트 아님) — 판정 분류는 그대로(확정위반 0 · 확정무위반 53 · 보류 1,094 · 종착 모두 보류)이고 바뀐 행은 예측대로 178 이다. 기록 문장에 질문이 선 행 178 → 0 · 보류 사유 「확신 부족」 987 → 825 · 「품목 선택 필요」 122 → 296.

- **불변식 · 경로 동치 측정**(원장 10-21(A) ② ~ ④ · 클론 A 회사 기기 · 커밋 `04b34cf`(②는 커밋 직전 같은 작업 트리) · 게이트 아님) — 불변식 위반 행 0(조건부 · 무조건부 · 품목만) · 같은 문장을 「전제를 고른 판정」과 「품목만 준 판정의 그 전제 분기」로 맞댄 경로 동치 어긋남 0 / 668 행.
- 전제마다 **기준 문안** 한 줄이 붙는다. 문안이 확정되지 않은 전제가 있으면 분기를 내지 않는다(`criteria_ready`).
- **2026-10-21 기준 화면의 흐름**(ksr 브랜치 코드 · main 병합 전) — 첫 검수는 품목 없이 판정해 분기 미리보기를 보인다. 사용자가 품목 · 전제를 고르거나 질문에 답하면 고른 전제의 제품 정보와 답(사실 · 실증)을 실어 그래프를 다시 부른다(`app/routers/user.py` `_review_pick` · `review_answer`). 결정기록은 보충 34의 화면 쪽 집행(이어 묻기 · 고른 것을 폼으로 받아 이력에 적기)을 아직 「대기」로 적고 있다.
- 계약에는 있으나 그래프가 읽지 않는 칸 — `has_recognized_function` · `recognition_no`(자격 칸은 고쳐 쓰기 서버가 전제를 알아야 할 때를 위해 채운다). 법별 노드가 고른 단서(`provisos`)는 판정이 단서를 어떻게 확인할지 정해지지 않아 응답에 일부러 싣지 않는다.

7. 상태 — 그래프가 나르는 것

상태	담는 것	그래프
`CoreState`	입력(원고 `text` · 제품 정보 `product`) · 문장 `sents` · 보낼 법 `laws` · 누적(검색 근거 `evidence` · 사전 적중 `dict_scans` · 인코더 신호 `encodings` · 법별 결과 `law_results` · 문장 판정 `sentences` · 노드별 시간 `timings`) · 분기 `branches` · 접힌 분기 `folded` · 참고 범위(`annex_scope` · `body_scope`) · 조문 원문(`basis_texts` · `ref_texts`)	코어
`ReviewState`	코어 + 종착 `outcome` · 증명서 `certificate` — 재생성 키(`attempt` · `rejects`)가 없다	검수
`GenerateState`	세그먼트 · 제품 정보 · 키워드 선별 · 후보 · 시도 횟수 · 거부 사유 · 이번 시도 거부 여부 · 매체 프로파일 · 각색본 · 종착 · 시간	생성 (스텝)
`ComposeState`	광고 종류 · 지면 구역	조립 (그래프 없음)

- **누적 키의 표는 한 곳이다**(`STATE_REDUCERS`) — 코어 · 검수는 `evidence` · `dict_scans` · `encodings` · `law_results` · `sentences` · `timings` 여섯. 팬아웃한 노드들이 같은 키에 쓰므로 리듀서가 없으면 마지막 것만 남는다. `sentences`만 덧붙이지 않고 같은 문장 id를 바꿔 끼우는 리듀서(`upsert_sentences`)를 쓴다 — 뒤 노드가 문장 판정을 고쳐도 문장 수가 늘지 않는다.
- **인코더 신호**(`SentEncoding`)는 문장 id · 유형별 확률 · 후보 · 잘림 여부 · 품목 입력을 나른다. 그래프 내부 운반체라 계약에 두지 않는다.
- **계측** — 노드마다 걸린 시간을 `timings`에 쌓는다(`@timed`). 외부 추적 도구를 쓰지 않으므로(D-43) 이 칸이 단계별 응답시간의 유일한 경로다. 코어를 부르는 함수 노드에는 두르지 않는다(두 번 세지 않게).
- **응답 계약**은 `JudgeResponse`(`app/contracts.py`) — 계약이 조용히 바뀌면 게이트(`tests/test_contract_surface.py`)가 잡는다. 10-21에 `ProductContext.premise` 칸이 더해졌고, 전제와 품목이 어긋나면 계약이 요청을 거부한다.
- **보류 종류**(10-21 보충 36 결정 4) — 문장 판정에 계산 칸 `hold_kind`가 실린다. 답 대기(문장에 질문이 섬) · 고르기 대기(품목 · 전제 보류) · 확인 못 함(확신 부족이지만 후보가 있음) · 걸린 것 없음(확신 부족이고 후보도 없음) · 그 밖 다섯이고, 이 순서대로 앞의 것이 이긴다(`app/contracts.py` `hold_kind` 한 곳). 응답에는 실리지만 저장 칸이 아니어서 DB · 사유 코드는 바뀌지 않았다. 어느 종류도 통과가 아니다.
- **두 겹 안내**(10-21 보충 37 결정 7) — 사용자가 고르는 칸(품목 · 전제 선택지 · 실증 질문 · 사실 확인 질문)마다 승인된 쉬운 풀이와 원문 조문을 붙인다(23 열쇠 · 조문 좌표 70 · `app/guide_table.py` · `app/guide_articles.json`). 응답을 다 만든 뒤 `guide.attach`가 붙이므로 판정 · 분기 · 위험도에는 들지 않는다.
- 두 칸 모두 2026-10-21 기준 화면은 아직 그리지 않는다. 계약 표면 시험(`tests/contract_surface.json`)에는 `HoldKind`가 더해졌다.

8. 설계 원칙이 구조에 박힌 자리

원칙	구조에서의 모양
없음이 성공으로 집계되지 않는다(D-220)	사전을 못 훑으면 미판정 · 검색을 못 하면 그 사실을 값으로 남김 · 문안이 없으면 증명서가 아니라 보류 · 빈 팬아웃 · 법별 결과 누락은 멈춤 · 인코더가 돌지 않았으면 카피 생성을 하지 않음
사전의 침묵은 통과가 아니다(D-269)	판정의 기본값이 보류 · 카피 생성도 「통과만 남김」이 아니라 「걸린 것을 떨굼」
위반을 확정하면 근거 조문이 붙는다(D-224)	계약의 검증기가 근거 없는 확정을 거부한다 · 인코더 근거도 대응표의 호로 근거를 세운다
답하지 않은 것을 가정하지 않는다(보충 33)	조건부 위반은 답 대기 보류 · 가장 보수적인 쪽은 위험도에만
검수는 문구를 만들지 않는다(D-265)	검수 그래프에 루프가 없고 후보 목록은 비어 있다 · 고쳐 쓰기 · 카피 생성은 그래프 밖 호출이고 그 결과를 같은 판정 함수로 다시 판정한다
판정 코어는 하나 · 같은 로직은 한 곳(D-99 · D-119)	검색 코어 하나 · 사전 매칭 하나 · 판정 함수 하나를 화면 · 생성 엔진 · 평가 도구가 같이 부른다
판정 경로에 외부 LLM을 쓰지 않는다(D-78 · D-332)	GPT는 카피 생성 후보를 쓰는 데만 · 판정은 사전 · 자체 인코더 · 규칙
판정 문구는 승인받은 글자만	기준 문안 · 사유 문안이 승인본과 글자가 같은지 게이트가 본다
잠금된 모델만 쓴다(D-333 · D-335)	인코더는 `models.lock.json` 의 잠금 판만 올리고(`enc.checked_dir`) 판 지문을 응답 표지에 남긴다 · sLLM 판도 같은 잠금 파일에 등재돼 있다

9. 응답시간 — 구조가 정하는 것

- 판정 목표는 p95 3초 미만이다(D-77). 원장에는 인코더가 붙은 뒤 이 목표로 잰 p95가 아직 없다(배포 서버의 측정은 아래 배포 담당 작업기록에 있다).
- **최신 측정**(원장 10-20 ① · 팀장 집 기기 Windows · `scripts/latency_probe.py` · 문구 「면역력 강화에 도움을 줍니다.」 · 품목 모름 · 예열 뒤 3번 중앙값 · D4-20 · 게이트 아님) — 판정 요청 한 번 **542 ms**(DB 연결 15 ms · 그래프 실행 522 ms). 노드별로 `retrieve` 291 · `encode` 115 · `match_dict` 104 · `references` 15 ms. 첫 요청은 모델 로드가 들어가 26,744 ms였다.
- 고치기 전에는 DB 호스트를 `localhost`로 적어 IPv6를 먼저 시도하느라 연결에만 3,130 ms가 걸렸다(요청 3,674 ms). 호스트를 `127.0.0.1`로 바꿨다.
- 구조가 정하는 비용 — 검색이 여전히 가장 크고(위 측정에서 그래프 실행의 절반 남짓), 문장 수에 비례한다. 인코더는 품목 입력이 여럿이어도 한 배치로 돌린다. 품목 분기는 전제마다 판정을 다시 계산하므로 품목을 모르면 전제 여섯을 모두 계산한다(이 부분만 따로 잰 값은 없다).
- 10-02의 옛 측정(원장 10-02 ⑧ · 클론 A 기기 · `scripts/time_graph.py` · 문장 10개 p95 4,350 ms · 20개 6,547 ms)은 인코더 · 위험도 · 품목 분기가 없던 코드의 수라 지금 구조의 수로 쓰지 않는다. 그때도 시간의 91 ~ 98 %가 `retrieve`였다.
- 평가 실행의 행당 시간은 다른 측정과 겹쳐 돌아 응답시간으로 쓰지 않는다.
- **배포 서버 측정**(원장 절이 아니라 배포 담당 소성민의 작업기록 · `docs/ssm/배포/작업기록_1019.md` · origin/ssm 브랜치, 아직 ksr에 없음 · 2026-10-19 · EC2 `g4dn.xlarge` · RDS `db.t4g.small` · 배포 커밋 `1f16723` · 인코더 D4-20 · sLLM 없음 · uvicorn 워커 1 · `load_test.py judge`) — 서버 안에서 동시 1 p50 0.52 s · p95 0.58 s, 동시 10 p50 2.73 s · p95 3.79 s, 동시 30 p50 9.03 s · p95 11.64 s이고, 처리량은 초당 약 3.3건에서 멈췄다. 밖(PC → Caddy)에서는 동시 1 p50 0.56 s · 동시 10 p50 3.05 s. 도메인 · HTTPS 단계의 부하 시험과 서버의 sLLM 한 문장 시간은 아직 재지 않았다.
- 카피 생성은 GPT 호출 · 라운드마다 후보 20개 판정이 들어가 수십 초가 걸린다(엔진 주석) — 화면이 스레드로 돌린다. 이 시간의 측정값은 원장에 없다.

10. 남은 것

1. **문서 단위 규칙(R-D1)** — `doc_rules` 노드의 원래 자리(문장 판정의 집합 연산 · 의무 고지문 등)는 비어 있다.
2. **품목 판별** — `classify`가 문구로 품목을 판별하는 일(보충 18 결정 1) · 미집행.
3. **인코더 쪽 남은 안건** — 거짓 · 과장의 확정 기준값(닫힘) · 주장 가드 역할 · D4-20 세 유형 기준값의 남은 확인(D-349 · 정하지 않은 것). 섞인 유형(한 문장에 인코더 근거 유형이 둘 이상)은 「유형마다 그 유형의 질문을 묻고 답을 다 받은 뒤 더 나쁜 쪽으로 합친다」로 정했고 집행은 봉인 뒤다(보충 37 결정 1). 10-21 ⑤의 새 틀린 확정 3은 규칙을 바꾸지 않고 알려진 오탐으로 적었다(보충 36 결정 3). 조건부 틀린 확정 4는 모두 특수용도식품 해설서 묶음에서 나왔고 그 묶음에 따로 전제를 두지 않기로 했다 — 확신 기준값은 봉인 뒤 다시 고를 때 함께 본다(보충 37 결정 4 · 원장 10-21(A) ⑤). 기능성화장품_오인은 확장 dev 양성이 5개뿐이라 측정 불가다.
4. **화면 쪽 집행** — 이어 묻기 · 고른 전제를 폼으로 받아 이력에 적기(보충 34 결정 2 · 4) · 보류 종류(`hold_kind`)와 두 겹 안내 그리기(§7) · E2E 실패 27(`일반식품_기능성` 분기 미표시 · 원장 10-21 ⑤). 이력은 「다시 판정하면 앞 기록을 지우지 않고 새 판정 행으로 쌓고, 홈 통계는 같은 사람 · 같은 문구의 가장 최근 판정만 센다」로 정했으나(보충 36 결정 2) 코드 반영은 대기다 — 지금 코드는 같은 문구의 최근 기록을 지우고 바꿔 적는다(`app/judgment_store.py` `replace_review`).
5. **생성 그래프(진입점 B)** — 스텁을 채울지, 진입점 B를 화면의 카피 생성 엔진으로 정의를 고칠지 판정 대기 · 코어 `POST /generate` · `/compose`는 501.
6. **고쳐 쓰기** — 재판정에 앞서 한 답도 넘기기로 정했고 서버 쪽 칸은 들어갔다(보충 37 결정 5). 화면에서 답을 그 칸으로 넘기는 배선은 대기다(§5-1). 그 밖에 sLLM 배포 포함 여부 · q4b-n2 잠금 등재 · 다듬기 규칙판(D-342)의 채택 조건 집행이 남았다.
7. **검색 · 리랭커** — 리랭커는 봉인 평가 뒤로 미뤘다(보충 30) · 법별 근거 수 `LAW_TOP_K`(5)는 임의 값이고 판정 대기(D-291).
8. **봉인 평가** — 그래프 · 인코더 평가 도구가 쓰는 봉인 평가셋(재분할 뒤 1,141 행 · D-347 결정 2)은 열지 않았다. 재분할 전의 옛 봉인 2,489 행은 10-14에 인코더 단독으로 한 번 쟀고 그 수는 보고 전용이다(D-339). 봉인의 범위와 조건은 정했다(보충 36 결정 1) — 봉인 조건은 봉인할 커밋과 같은 커밋의 dev 에서 불변식 위반 행 0 · 경로 동치 어긋남 0 이다(`04b34cf` 측정에서는 둘 다 0 · §6). 봉인 전에는 측정 방법과 문구로 끝나는 안건만 고치고, 섞인 유형 · 주장 가드 · 거짓 · 과장 기준값 · 품목 판별 · 알려진 오탐 등은 봉인 보고에 「알려진 한계」로 적는다.
9. **응답시간** — 도메인 · HTTPS 단계의 부하 시험 · 서버의 sLLM 한 문장 시간 · 배포 서버 측정(10-19 · 배포 담당 작업기록)의 원장 등재.

> **근거** — 코드 `app/graph.py`(`CORE_BEFORE_LAWS` · `CORE_AFTER_LAWS` · `LAW_SCOPE` · `route_review` · `build_core` · `build_review` · `build_generate`) · `app/contracts.py`(`hold_kind`) · `app/premise.py` · `app/guide.py` · `app/judgment_store.py` · `app/api.py`(`judge`) · `app/routers/user.py`(`_core_judge` · `_rejudge` · `_rewrite_sentence` · `_review_pick` · `review_answer` · `_lands`) · `app/routers/sllm_client.py` · `docs/lse/persona_pipeline_e2e.py` · `docs/lse/polish_rule.py` · `app/routers/copygen/`(`__init__` · `pipeline` · `judge` · `filters` · `llm` · `branches`) · `models.lock.json` · 수치 원장 10-02 ⑧ · 10-19 ④ · ⑩ · 10-20 ① · 10-21 ⑤ · 10-21(A) ② ~ ⑥ · 결정기록 D-11 · D-43 · D-59 · D-77 · D-78 · D-99 · D-119 · D-125 · D-220 · D-224 · D-263 · D-265 · D-266 · D-267 · D-268 · D-269 · D-271 · D-277 · D-291 · D-319 · D-323 · D-332 · D-333 · D-335 · D-338 · D-341 · D-342 · D-345 · D-347 · D-348(보충 2 · 5 · 6 · 8 · 16 · 18 · 20 · 23 · 30 · 31 · 32 · 33 · 34 · 35 · 36 · 37) · D-349 · 상태 요약 `docs/02_설계/LangGraph_상태_스키마.md`